KOI Content Pack

Troubleshooting & Data Provenance

For Cortex XSIAM — Windows endpoints

Every finding in this guide was verified on a live tenant and a live Windows endpoint. Commands shown are the ones that actually worked, including the ones that silently do not.

Document	KOI Content Pack — Troubleshooting & Data Provenance
Applies to	Cortex XSIAM, KOI on Windows (Server 2022 verified)
Pack version	1.3.0
Verified on	win-workstation, 20 July 2026

Table of Contents

1. How the Pieces Actually Fit Together

Most KOI troubleshooting goes wrong because of one incorrect assumption: that a KOI agent runs continuously on the endpoint. On Windows it does not.

1.1 There is no resident KOI agent

On a live Windows endpoint we checked for all three ways software can persist, and found none of them:

Checked	Result
Windows service named *koi*	None. sc query returns nothing.
Running process named *koi*	None in tasklist.
Scheduled task named *koi*	None in schtasks.

KOI on Windows is run-on-demand. The Cortex agent executes the KOI deployment script; the script scans, uploads its findings, refreshes its configuration and exits. Between runs nothing KOI is resident.

Consequence: the XSIAM Job is the scan scheduler. No Job run means no scan — not a delayed scan, not a partial scan. Nothing. Any investigation into stale KOI data should start with the Job, not the endpoint.

1.2 The end-to-end chain

Every data point you see in KOI traverses this chain. Knowing which link you are looking at tells you which tool to reach for:

```
XSIAM Job  ->  KOI Script Runner - Scan Job  ->  core-script-run
       ->  KOI deployment script on the endpoint
       ->  reads filesystem / registry / browser profiles
       ->  HTTPS POST to the KOI backend
       ->  koi-* integration commands read it back
```

A break at any link presents identically from the console: KOI data looks stale. Sections 5 and 6 give a test for each link.

2. The Three Diagnostic Channels

Three independent ways to interrogate the same endpoint. Using more than one is what turns a guess into a diagnosis, because each sees a different link in the chain.

Channel	What it shows	Use it when
KOI integration	What the KOI backend believes, after the last successful scan	Confirming data reached KOI
XSIAM core	The live endpoint, via the Cortex agent, running as SYSTEM	Confirming the endpoint's real state without leaving XSIAM
Direct PowerShell	The raw filesystem and registry, as an interactive admin	Establishing ground truth to compare the other two against

2.1 Channel A — the KOI integration

Runs from any war room or the playground. Answers the question "what does KOI think is installed?"

```
!koi-devices-list limit=50
```

```
!koi-device-inventory-get device_id=<koi-device-id> limit=50
!koi-inventory-item-get item_id=<id> marketplace=<mp>
```

Note: the KOI device id is not the Cortex endpoint id and not the hostname. Find it in agent_policies.json on the endpoint (section 3.2), or by matching hostname in koi-devices-list.

2.2 Channel B — XSIAM core commands

Runs arbitrary commands on the endpoint through the Cortex agent. Requires only that the endpoint is CONNECTED — no credentials, no network path, no open ports.

```
!core-run-script-execute-commands endpoint_ids="<endpoint_id>" commands="<command>"
timeout=600
```

The command returns an action_id, not the output. Fetch the output separately:

```
!core-get-script-execution-results action_id="<action_id>"
```

2.3 Channel C — direct PowerShell over SSH

For a GCP-hosted Windows endpoint reached through Identity-Aware Proxy. Establish the tunnel, then connect:

```
gcloud compute start-iap-tunnel <instance> 22 --local-host-port=localhost:2222 --zone
<zone>
ssh -i ~/.ssh/<key> -p 2222 <admin-user>@localhost
```

Three prerequisites, each of which we hit as a real failure — see section 7.3 for the diagnosis:

1. An SSH server must be installed and running on the guest.
2. The public key must be in C:\ProgramData\ssh\administrators_authorized_keys, with inheritance removed and access granted only to Administrators and SYSTEM. sshd silently ignores the file otherwise.
3. The instance must carry the network tag the IAP firewall rule targets. A rule scoped to a tag does nothing for an instance without it.

3. KOI on Windows — the File Map

3.1 Everything lives in one directory

`C:\ProgramData\Koi\` — there is no Program Files install.

File	Typical	Purpose
settings.json	~10 KB	Scan configuration pulled from the backend: every collector module and whether it is enabled — browsers, package managers, IDE, Office, installed software, AI tooling, and the remediation actions.
agent_policies.json	~1 KB	Enforcement policy set: policies with target agents, enforcement mode and rules, plus exclusions, the approval URL and the block message template shown to the user.
agent_activity.jsonl	0 B when idle	Append-only JSON Lines record of agent activity.
agent_enforcement.log	0 B when idle	What the enforcement engine evaluated and blocked.
koi_agent_enforcement	symlink	Points at the active versioned module directory. Repointing this symlink is how an upgrade takes effect.
koi_agent_enforcement_v<hash>	directory	Versioned enforcement runtime. Multiple versions are

File	Typical	Purpose
		retained; the previous one stays for rollback.

Both log files being zero bytes is normal on a host where nothing has been blocked. It is not evidence of a broken agent.

3.2 The two fields you will need

agent_policies.json carries the identifiers that tie this endpoint to your KOI tenant:

Field	What it is
approval_url -> cid	Your KOI customer/tenant id.
approval_url -> deviceId	The KOI device id for this endpoint — the value koi-device-inventory-get expects.
mdm_version	Version of the policy schema the agent is running.

3.3 KOI ships its own Python

A bundled WinPython runtime is installed under the Default user profile. It is reachable by two paths that resolve to the same place — the second is a junction:

```
C:\Users\Default\AppData\Local\Koi\Python\WPy64-31290\python\python.exe
C:\Users\Default\Local Settings\Koi\Python\WPy64-31290\python\python.exe
```

This matters for interpreting inventory. PyPI packages can appear in KOI on a host where nobody ever installed Python, because KOI is partly inventorying its own interpreter. Do not treat that as a false positive or as shadow IT.

4. How Each Data Point Is Collected

This section traces individual data points from disk to the KOI console. It exists so that when a value looks wrong, you can tell whether KOI is wrong or your comparison is.

4.1 Browser extensions

Source of truth on disk, per user profile:

```
C:\Users\<user>\AppData\Local\Google\Chrome\User
Data\Default\Extensions\<id>\<version>_0\manifest.json
```

The scanner walks every user profile, then for each extension reads the id from the directory name, the version from manifest.json, and the display name as described below.

The display name is usually not in manifest.json. Chrome stores a localization placeholder there instead. Verified on a live host, three of four extensions resolved their name from a locale file, not the manifest:

Extension	Version	Where the display name came from
Google Translate	2.0.16	_locales\en\messages.json, key 8969005060131950570
Google Docs Offline	1.106.1	_locales\en\messages.json, key extName
Fireflies: AI meeting notes	6.3.1	manifest.json:name (literal — the exception)
Chrome Web Store Payments	1.0.0.6	_locales\en\messages.json, key APP_NAME

So a manifest.json whose name reads __MSG_8969005060131950570__ is not corruption. KOI resolves it through _locales\<lang>\messages.json exactly as Chrome does. Comparing KOI's display name against manifest.json:name alone will produce a false mismatch.

4.2 Installed software

Read from the registry uninstall hives. Both must be read — querying only the first misses every 32-bit application on a 64-bit host:

```
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*
HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*
```

Only entries carrying a DisplayName are inventory candidates.

4.3 Code packages

PyPI and npm inventory comes from the package metadata of the interpreters and package managers present on the host — including KOI's own bundled Python described in section 3.3.

4.4 Configuration and policy

settings.json and agent_policies.json are not written locally by the scan. They are pulled from the backend on each run, which makes their modification time the single most useful signal on the endpoint — see section 6.

5. Verifying That a Scan Completed

Three independent checks, one per link in the chain. Run them in order; the first one that fails localises the fault.

5.1 Check 1 — did XSIAM dispatch and complete the script?

In the Job run's war room, the per-entry result should carry ok:true and an action_id. Then confirm the action itself:

```
!core-get-script-execution-results action_id="<action_id>"
```

Field	Expected on success
execution_status	COMPLETED_SUCCESSFULLY
_return_value	0
failed_files	0
endpoint_status	STATUS_010_CONNECTED

5.2 Check 2 — did the script run on the endpoint?

The on-box proof. Both files should carry a modification time equal to the run:

```
dir C:\ProgramData\Koi
```

If check 1 passed but these timestamps are old, the script was dispatched and reported success without completing its work — look at the script itself, not at XSIAM.

5.3 Check 3 — did the data reach KOI?

The authoritative check. Using the deviceId from agent_policies.json:

```
!koi-device-inventory-get device_id=<deviceId> limit=50
```

Every returned item carries a Last Seen. On a healthy host that timestamp matches the run time from checks 1 and 2. If checks 1 and 2 passed and this is stale, the endpoint could not reach the backend — go to section 6.

6. Verifying Backend Connectivity

Work outwards from the endpoint. Each step isolates a different failure.

6.1 Name resolution

```
nslookup <your-koi-backend-host>
```

The KOI backend is fronted by a CDN, so expect the answer to resolve to a CDN hostname and address rather than to KOI directly. That is normal.

6.2 TLS and HTTP

Avoid Test-NetConnection through the XSIAM channel — see section 7.4. Use curl and write the headers to a file:

```
curl.exe -sS -m 20 -D C:\Windows\Temp\koihdr.txt -o NUL https://<your-koi-backend-host>/
type C:\Windows\Temp\koihdr.txt
```

Response headers prove DNS, routing, TLS and HTTP all work.

6.3 The check that actually matters

Reachability is not proof of a working agent. settings.json and agent_policies.json are only rewritten after the agent authenticates and successfully pulls its configuration. A fresh modification time on those two files is the only evidence that the complete round trip — including credentials and enrolment — succeeded.

Read the two together:

Observation	Conclusion
Headers return, config files fresh	Fully healthy.
Headers return, config files stale	Network is fine. Suspect enrolment, credentials or the script failing before its config fetch.
No headers	Network path problem: egress filtering, proxy, TLS interception or DNS.

7. Symptom, Cause, Fix

Every entry below was observed in practice, not imagined.

7.1 The Job runs but nothing is ever scanned

The most common failure, and the most misleading, because every run reports success and closes cleanly.

Symptom	Cause	Fix
Every entry reports SKIPPED, no failure email, run closes as Resolved	No connected, unisolated endpoint matches the entry's OS and group scope. This is the designed skipped	Compare group membership against connected status — see 7.2

Symptom	Cause	Fix
	state, deliberately silent	
Job never fires at all	Job disabled, or its schedule never reached	Check schedulingStatus and nextRunTime on the Job
Run parks in running for a long time	The script's own timeout is longer than the Job interval, so runs overlap	Lengthen the interval or shorten the script timeout

7.2 The endpoint targeting trap

A skipped run means the intersection of three conditions was empty. All three must hold simultaneously:

- The endpoint is a member of a group named in target.endpoint_groups
- Its status is CONNECTED — not DISCONNECTED and not LOST
- Its OS matches target.endpoint_os, and it is unisolated

The classic case, seen live: the only member of the target group was disconnected, while the two connected machines that could have run the script were not in the group. Both facts are individually unremarkable; together they produce a job that runs forever and does nothing.

```
!core-get-endpoints group_name="<group>" status="connected" isolate="unisolated"
```

An empty result here is the whole explanation. Note that the endpoint list in the console shows group membership and status in separate columns — it is easy to confirm one and assume the other.

7.3 Cannot get a shell on the endpoint

Symptom	Cause	Fix
IAP tunnel: failed to connect to backend, port 22	The firewall rule allowing IAP to 22 is scoped by network tag and the instance lacks that tag	Add the tag the rule targets
enable-windows-ssh metadata has no effect	The Google guest agent is what consumes that metadata. If it is stopped or disabled, the flag is inert	Check the guest agent's state before trusting any metadata-driven mechanism
Password reset does not work either	Same root cause — that path is also handled by the guest agent	Install and configure an SSH server directly instead
Key is present but sshd rejects it	administrators_authorized_keys must have inheritance removed and grant only Administrators and SYSTEM	icacls <file> /inheritance:r /grant Administrators:F /grant SYSTEM:F

7.4 The tooling lies to you

Three behaviours that produce confidently wrong answers. Each cost real investigation time.

Behaviour	What to do instead
powershell -Command - executes piped stdin line by line, so multi-line foreach and if blocks break apart. You get a header, no rows, and no error	Use -EncodedCommand with base64 UTF-16LE. This is the single most important tip in this guide: it turns silent wrong answers into correct ones
The XSIAM script channel mangles pipe characters and nested	Return the unfiltered output and filter it locally

Behaviour	What to do instead
quotes, so findstr filters return nothing and appear to prove absence	
Get-ChildItem with -ErrorAction SilentlyContinue hides access and path errors, so a permissions problem looks like an empty directory	Drop the suppression while diagnosing, and confirm with Test-Path

7.5 Duplicate device records in KOI

A host that has been re-enrolled can hold more than one record in KOI: an old one under the hostname, stale for weeks, and a current one under the device id the agent now reports.

Searching KOI by hostname finds the stale record and leads to exactly the wrong conclusion. Always verify with the deviceId taken from agent_policies.json on the endpoint itself. If the id-based lookup is fresh and the hostname-based one is stale, the scan is working and the old record needs cleaning up.

8. Command Reference

8.1 XSIAM — Job and endpoint state

```
!core-get-endpoints group_name="<group>" status="connected" isolate="unisolated"
!core-run-script-execute-commands endpoint_ids="<id>" commands="<cmd>" timeout=600
!core-get-script-execution-results action_id="<action_id>"
```

8.2 KOI — what the backend believes

```
!koi-devices-list limit=50
!koi-device-inventory-get device_id=<deviceId> limit=50
!koi-users-list limit=1
```

8.3 On the endpoint

```
dir C:\ProgramData\Koi
type C:\ProgramData\Koi\agent_policies.json
sc query sshd
nslookup <koi-backend-host>
curl.exe -sS -m 20 -D C:\Windows\Temp\koihdr.txt -o NUL https://<koi-backend-host>/
```

8.4 Running PowerShell reliably over SSH

Encode the script rather than piping it, for the reason given in 7.4:

```
python3 -c "import base64;print(base64.b64encode(open('s.ps1').read().encode('utf-16-le')).decode())"
ssh -i <key> -p 2222 <user>@localhost "powershell -NoProfile -EncodedCommand <base64>"
```

8.5 Fast triage order

1. Is the Job enabled and did it run? Check schedulingStatus and lastRunTime.

2. Did each entry return ok:true, or SKIPPED? SKIPPED means targeting — go to 7.2.
3. Did the action complete? COMPLETED_SUCCESSFULLY with return value 0.
4. Are the endpoint's config files fresh? dir C:\ProgramData\Koi.
5. Is KOI's inventory fresh for the deviceId from agent_policies.json?
6. If not, test the backend path — section 6 — and remember reachability alone proves nothing.